DATABASE MANAGEMENT SYSTEMS | 23A05402T

UNIT – I : Introduction & Entity Relationship Model

Easy handwritten notes with diagrams for students

Topics in this unit

1. Database system, characteristics (Database vs File system)
2. Database users, advantages and applications of DBMS
3. Data models, schema, instance and data independence
4. Three-schema architecture, DBMS structure, environment
5. Centralized and Client-Server architecture
6. ER model: entities, attributes, relationships, constraints
7. Sub classes, super class, inheritance, specialization, generalization

* **Study tip:** Every topic has a small diagram or table. Learn the **RED words** first, then draw the diagram once from memory - that is the fastest way to remember it for exams.

Syllabus (as given)

Introduction: Database system, Characteristics (Database Vs File System), Database Users, Advantages of Database systems, Database applications. Brief introduction of different Data Models; Concepts of Schema, Instance and data independence; Three tier schema architecture for data independence; Database system structure, environment, Centralized and Client Server architecture for the database. Entity Relationship Model: Introduction, Representation of entities, attributes, entity set, relationship, relationship set, constraints, sub classes, super class, inheritance, specialization, generalization using ER Diagrams.

Part 1 : Introduction to Database Systems

1.1 What is a Database System?

Definition: **Data** = raw facts (e.g. 101, Ravi, 20). **Information** = processed data with meaning. A **database** is an organised collection of related data stored so that it can be easily accessed, managed and updated. A **DBMS** (Database Management System) is the software that creates, stores, retrieves and controls the database. Database + DBMS + Application programs = **Database System**.

- Goal 1: **Store** large data safely and permanently.
- Goal 2: **Retrieve** it quickly using queries (e.g. SQL).
- Goal 3: **Share** data among many users with security and correctness.
- Examples of DBMS: Oracle, MySQL, PostgreSQL, SQL Server, MongoDB, IBM DB2.

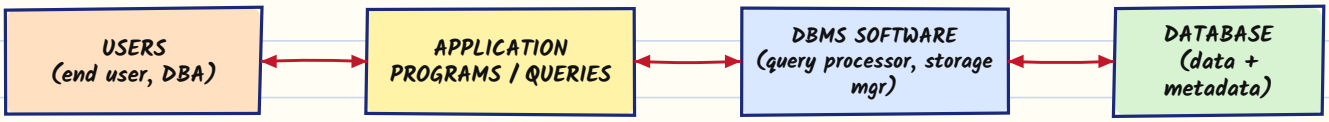


Fig 1.1 : Components of a database system

* **Remember:** Metadata = 'data about data' (table names, column types, constraints). It is stored in the **system catalog / data dictionary**.

1.2 File System vs Database System (Characteristics)

Before DBMS, data was kept in separate files (one per application). This caused many problems, which a DBMS solves.

Point	File system	Database system (DBMS)
Redundancy	Same data repeated in many files	Controlled - data stored once
Consistency	Copies may disagree (inconsistency)	Single copy keeps data consistent
Data access	New program needed for each query	Query language (SQL) gives ad-hoc access
Data isolation	Data scattered in different formats	Common structure and format
Integrity	Rules coded inside every program	Constraints declared once in the DBMS
Concurrency	Simultaneous updates cause errors	Concurrency control handles many users
Security	Weak, OS-level only	User accounts, privileges, views
Recovery	Hard to restore after crash	Backup + log-based recovery
Atomicity	Half-done updates remain	Transaction is all-or-nothing

Main characteristics of the database approach

- **Self-describing** - catalog stores the structure of the database.
- **Insulation** between programs and data (program-data independence).
- **Multiple views** - each user sees only the part he needs.
- **Data sharing** and multi-user transaction processing.

- **Controlled redundancy**, integrity constraints, security and backup/recovery.

1.3 Advantages and Disadvantages of DBMS

Advantages	Disadvantages
Reduces redundancy and inconsistency	High initial cost (software, hardware, training)
Data sharing among users and applications	Complexity - needs skilled DBA
Enforces integrity and security	Performance overhead for very small applications
Backup and recovery, concurrency control	Single point of failure if not replicated
Data independence, faster development	Upgrades / conversion can be costly

1.4 Database Users

User type	Role
Database Administrator (DBA)	Controls the whole DB: schema definition, storage, security, backup, performance tuning, grants permissions.
Database Designer	Decides what data to store and designs the structure (ER diagram, tables).
Application Programmer	Writes programs (Java, Python, PHP) that access the DB using SQL.
Sophisticated user	Analysts / scientists who write their own complex queries directly.
Naive (end) user	Uses ready-made forms/apps, e.g. bank clerk, ticket booking user.
Casual user	Occasional user, e.g. manager who sometimes queries the DB.

1.5 Database Applications

Area	Use of database
Banking	Accounts, transactions, loans, ATM
Railways / Airlines	Reservation, schedules, tickets
Universities	Student records, results, attendance, library
E-commerce	Products, orders, payments, customers (Amazon, Flipkart)
Telecom / Social media	Call records, bills, user profiles, posts
Healthcare / Government	Patient records, Aadhaar, tax, voter lists

Part 2 : Data Models, Schema and Architecture

2.1 Data Models (brief introduction)

A **data model** is a collection of concepts used to describe the structure of a database (data, relationships, constraints). It hides storage details.

Data model	Simple explanation
Hierarchical	Data in a tree - one parent, many children (IBM IMS). Fast but rigid.
Network	Graph - a record can have many parents (CODASYL). Flexible but complex.
Relational	Data in tables (rows and columns) linked by keys. Most popular (MySQL, Oracle).
Entity-Relationship (ER)	High-level conceptual model using entities and relationships; used for design.

Data model	Simple explanation
Object-oriented	Stores data as objects with attributes and methods (db4o).
Object-relational	Relational model + object features (PostgreSQL, Oracle).
NoSQL / Semi-structured	Document (MongoDB, JSON), key-value (Redis), column, graph (Neo4j) - for big, flexible data.

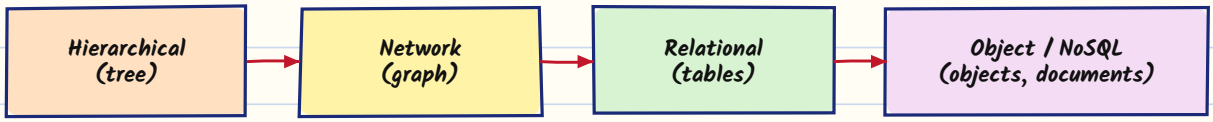


Fig 1.2 : Evolution of data models

2.2 Schema and Instance

Term	Meaning	Changes?
Schema	The overall design / structure of the database (table names, columns, types, constraints). Like the blueprint of a building.	Rarely
Instance	The actual data stored in the database at a particular moment (snapshot). Like the people living in the building.	Very often

Example schema: **STUDENT(Roll_no, Name, Age, Dept)**. Example instance: **(101, Ravi, 20, CSE), (102, Sita, 19, ECE)**.

Types of schema: **Physical schema** (storage), **Logical (conceptual) schema** (tables, relations) and **External schema / subschemas** (user views).

2.3 Three-Schema (Three-Tier) Architecture

Proposed by ANSI-SPARC. It separates the database into 3 levels so that users are isolated from the physical details.

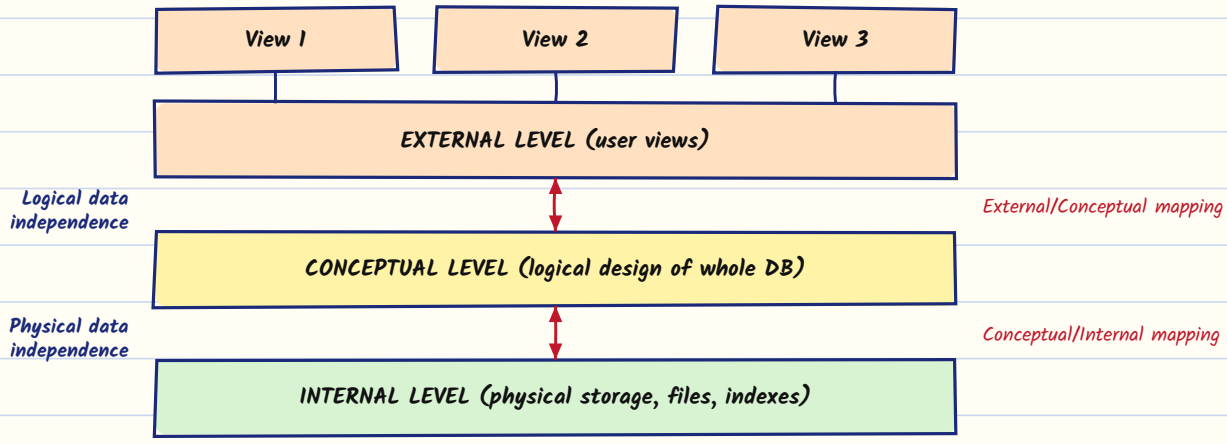


Fig 1.3 : Three-schema architecture and data independence

Level	Describes	Who uses it
External (view) level	Part of DB relevant to a user group; hides the rest	End users, application programmers
Conceptual (logical) level	Entities, attributes, relationships, constraints of the whole DB	DBA, designers

Level	Describes	Who uses it
Internal (physical) level	How data is stored: file organisation, indexes, access paths	DBMS / system programmers

2.4 Data Independence

Data independence = the ability to change the schema at one level without having to change the schema at the next higher level. It is the main benefit of the three-schema architecture.

Type	Meaning	Example
Physical data independence	Change internal (physical) schema without changing conceptual schema	Add an index, move file to a new disk, change file organisation
Logical data independence	Change conceptual schema without changing external views / programs	Add a new column or table; programs using old views still work

* **Remember:** Logical independence is **harder** to achieve than physical independence, because application programs depend strongly on the logical structure.

2.5 Database System Structure (Components of DBMS)

A DBMS has two main parts: the **Query Processor** and the **Storage Manager**, working on top of the disk storage.

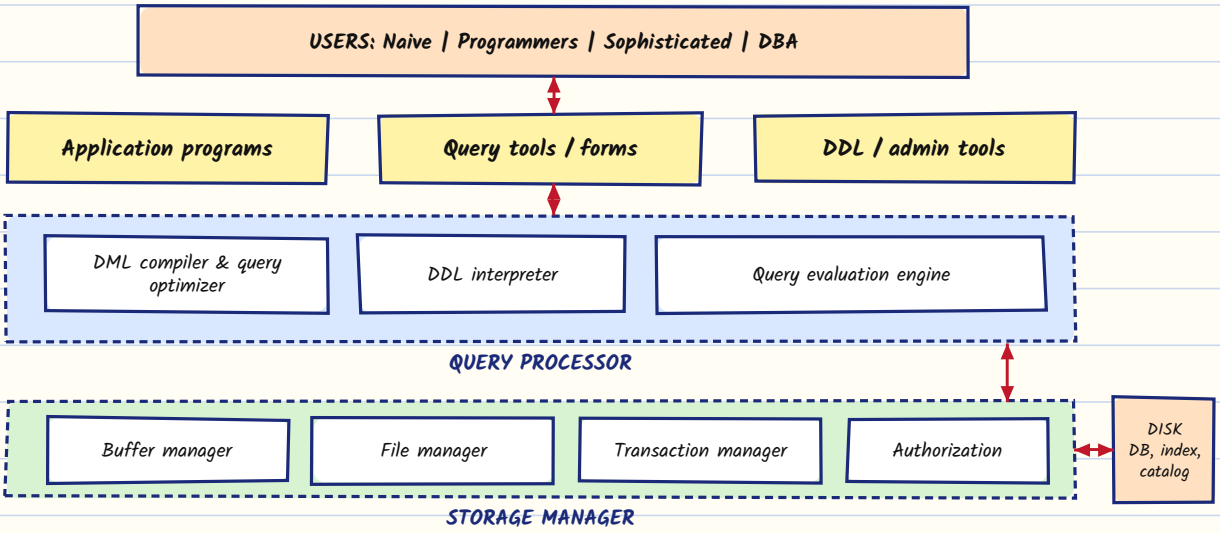


Fig 1.4 : Structure of a DBMS

Component	Job
DDL interpreter	Reads schema definitions and stores them in the catalog.
DML compiler / optimizer	Converts a query into an efficient low-level plan.
Query evaluation engine	Executes the plan and returns the result.
Buffer manager	Moves data between disk and main memory.
File manager	Manages space on disk and the data structures used.
Transaction manager	Ensures ACID properties - concurrency and recovery.
Authorization & integrity manager	Checks access rights and integrity constraints.

2.6 Database Environment and Languages

- **DDL** (Data Definition Language) - CREATE, ALTER, DROP: defines the schema.
- **DML** (Data Manipulation Language) - SELECT, INSERT, UPDATE, DELETE: handles data.
- **DCL** (Data Control Language) - GRANT, REVOKE: permissions.
- **TCL** (Transaction Control Language) - COMMIT, ROLLBACK, SAVEPOINT.
- Environment = DBMS software + hardware + data + users + procedures (+ network).

2.7 Centralized and Client-Server Architecture

Centralized architecture

DBMS, data and applications all run on **one computer** (mainframe or PC). Users connect through dumb terminals. Simple, but the single machine is a bottleneck and a single point of failure.

Client-Server architecture

Work is split: the **client** runs the user interface and application logic; the **server** runs the DBMS and stores the data. They communicate over a network.

Two-tier

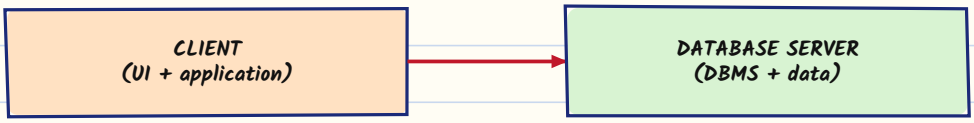


Fig 1.5 : Two-tier architecture (client talks to DB directly via ODBC/JDBC)

Three-tier



Fig 1.6 : Three-tier architecture (used in web applications)

Point	Centralized	Client-Server
Processing	All on one machine	Shared between client and server
Cost / scaling	Cheap but hard to scale	Scalable, add more clients/servers
Failure	Whole system stops	Only affected part stops
Example	Old mainframe banking	Web apps, ERP, online shopping

Part 3 : Entity Relationship (ER) Model

3.1 Introduction

The **ER model** (Peter Chen, 1976) is a high-level **conceptual** data model used to design a database before creating tables. It describes the real world as **entities**, their **attributes** and the **relationships** among them, drawn as an **ER diagram**.

Steps: requirements -> ER diagram -> convert to relational schema -> create tables.

3.2 Entity, Entity Set and Attributes

- **Entity** - a real-world thing that can be identified (a student, a book, a course).
- **Entity set** - collection of entities of the same type (all students). Shown as a rectangle.

- **Strong entity** - has its own key attribute. **Weak entity** - no key of its own; depends on an owner (strong) entity through an identifying relationship (e.g. DEPENDENT of EMPLOYEE).

Types of attributes

Attribute	Meaning	Example
Simple (atomic)	Cannot be divided	Age, Roll_no
Composite	Can be divided into parts	Name = First + Last; Address
Single-valued	One value per entity	DOB
Multi-valued	Many values per entity	Phone numbers, Skills
Derived	Calculated from other attribute	Age from DOB
Key attribute	Uniquely identifies an entity	Roll_no
Null attribute	Value unknown / not applicable	Middle name

ER diagram symbols

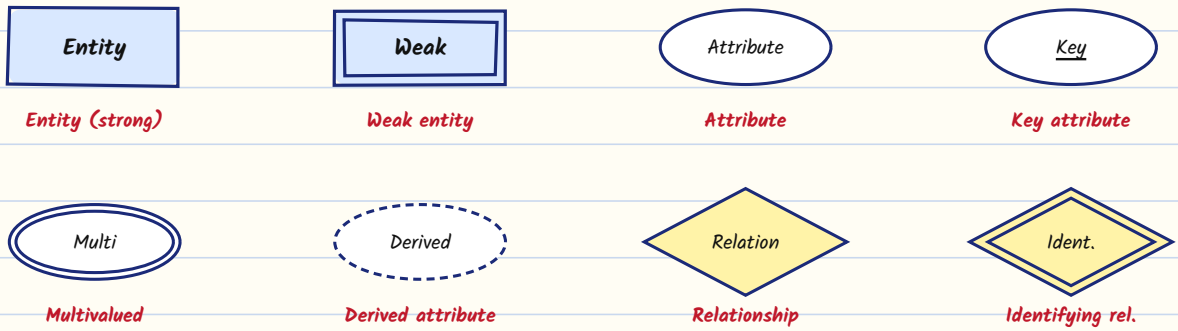


Fig 1.7 : Basic ER diagram notation

3.3 Relationship, Relationship Set and Degree

- **Relationship** - an association among entities (Student ENROLLS in Course).
- **Relationship set** - collection of relationships of the same type. Shown as a diamond.
- **Degree** - number of entity sets taking part: unary (1), binary (2), ternary (3).
- A relationship can have its own attributes (e.g. Grade in ENROLLS).

3.4 Constraints on Relationships

(a) Mapping cardinality (cardinality ratio)

Type	Meaning	Example
One-to-One (1:1)	One A is related to at most one B	Person - Passport
One-to-Many (1:N)	One A relates to many B; each B to one A	Department - Employee
Many-to-One (N:1)	Many A relate to one B	Employee - Department
Many-to-Many (M:N)	Many A relate to many B	Student - Course

(b) Participation constraint

- **Total participation** (double line) - every entity must take part in the relationship (every Employee works for a Department).
- **Partial participation** (single line) - some entities may not take part.

Example ER diagram : STUDENT enrolls in COURSE

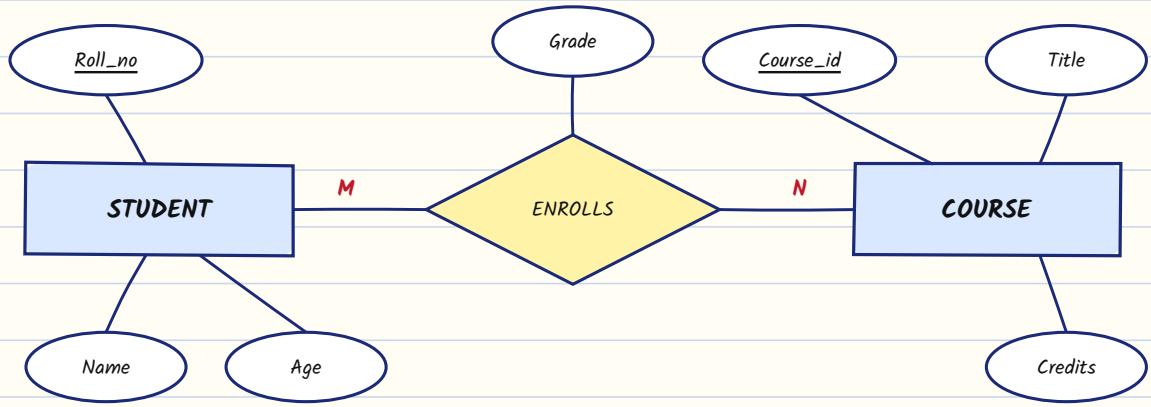


Fig 1.8 : ER diagram with M:N relationship and relationship attribute

3.5 Super Class, Sub Class, Inheritance

- **Super class** - the general entity type (EMPLOYEE).
- **Sub class** - a more specific type that is a subset (ENGINEER, SECRETARY). Relationship is called **IS-A**.
- **Inheritance** - a sub class automatically gets all attributes and relationships of its super class, plus its own extra attributes.

Specialization (top-down)

Start with a general entity and divide it into more specific sub classes based on some difference. EMPLOYEE -> ENGINEER, SECRETARY, TECHNICIAN.

Generalization (bottom-up)

Start with several specific entities and combine their common features into one general super class. CAR and TRUCK -> VEHICLE.

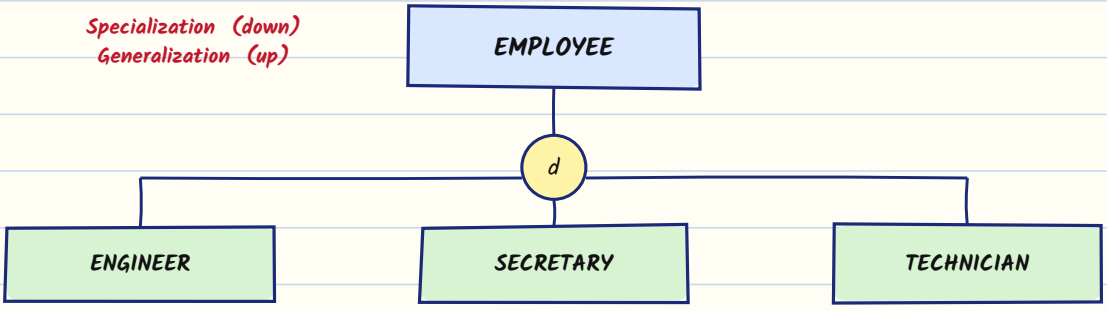


Fig 1.9 : Specialization / generalization (circle 'd' = disjoint, 'o' = overlapping)

Constraints on specialization

Constraint	Meaning
Disjoint (d)	An entity can belong to at most ONE sub class (a person is either Male or Female).
Overlapping (o)	An entity can belong to MORE than one sub class (a person can be both Student and Employee).
Total (double line)	Every super class entity must be in some sub class.
Partial (single line)	Some super class entities may belong to no sub class.

* **Remember:** Specialization = **top-down** (split). Generalization = **bottom-up** (merge). Both are shown with the same diagram; only the direction of thinking differs.

Quick Revision - Unit 1

- DBMS = software to create, store, retrieve and control a database; database = organised related data.
- File system problems: redundancy, inconsistency, isolation, weak security, no concurrency - DBMS solves these.
- Users: DBA, designer, application programmer, sophisticated, naive, casual.
- Data models: hierarchical, network, relational, ER, object-oriented, NoSQL.
- Schema = structure (rarely changes); Instance = data now (changes often).
- Three levels: External -> Conceptual -> Internal. Physical & logical data independence.
- DBMS parts: Query processor + Storage manager. Languages: DDL, DML, DCL, TCL.
- Architecture: centralized, 2-tier and 3-tier client-server.
- ER: entity (rectangle), attribute (ellipse), relationship (diamond); keys underlined.
- Cardinality 1:1, 1:N, M:N; participation total / partial; weak entity.
- Specialization (top-down), generalization (bottom-up), inheritance, disjoint/overlapping.

Possible exam questions

1. Define DBMS. Compare file system and database system.
2. Explain the advantages of DBMS and the different types of database users.
3. Explain the three-schema architecture. How does it give data independence?
4. Differentiate schema and instance; physical and logical data independence.
5. Explain the structure (components) of a DBMS with a neat diagram.
6. Explain centralized and client-server (2-tier, 3-tier) architectures.
7. Explain ER model concepts: entity, attributes (types), relationships, constraints.
8. Explain specialization, generalization and inheritance with ER diagrams.
9. Draw an ER diagram for a University / Library / Hospital database.